

Lab Course Report: "Make Car Go Fast"

Simon Wilhelm Schübel
Karlsruhe Institute of Technology
Karlsruhe, Germany
Email: uerga@student.kit.edu

Tahmur Hassan Mahmood
Karlsruhe Institute of Technology
Karlsruhe, Germany
Email: uxgzc@student.kit.edu

Abstract—This student project focuses on training a reinforcement learning agent for the game Trackmania Nations Forever. Building upon the foundations of the Trackmania-Gym framework, we extend its capabilities through the implementation of a headless, containerized training environment, vectorized multi-track training, and the transition from discrete to continuous action spaces. Notably, we provide the first demonstration of successful continuous control within the TMNF RL scene, showing that continuous agents perform on par with, and in some scenarios exceed, the performance of traditional discrete-action baselines. Furthermore, we introduce a novel integration of the SimbaV2 framework with Proximal Policy Optimization (PPO). Our results indicate that the SimbaV2-PPO architecture provides a significant performance boost over standard PPO. While infrastructure constraints limited our access to high-performance computing (HPC), this work establishes a robust, scalable architecture for future RL research in racing simulations.

I. INTRODUCTION

Reinforcement Learning (RL) is a powerful framework for learning decision-making policies in environments that require sequential reasoning. In particular, visual RL has demonstrated impressive results, achieving superhuman performance in games such as ATARI and racing simulations like Gran Turismo and Trackmania Nations Forever (TMNF) [45, 37]. TMNF, released by Nadeo in 2008, is a highly dynamic racing game characterized by complex track geometries, including sharp turns, jumps, loops, and wall-rides. In this study, we focus on the *Time Attack* mode, where the objective is to minimize lap time. This requires pixel-perfect driving, optimal pathfinding, and precise control of the car, which makes TMNF an interesting environment for RL research [35, 42]. Especially for autonomous systems, such as self-driving cars, the ability to learn from raw visual observations without relying on handcrafted rules is a significant advantage [23, 24, 51].

RL for TMNF has attracted a large community, both within the RL community and also for the TMNF community, who enjoy the progress of RL within TrackMania. Projects like Linesight Trackmania AI [1] had huge public exposure, bringing the concept of reinforcement learning within TMNF to millions of people on YouTube ¹.

This work extends a previously introduced gymnasium environment [62] for single-track training in TMNF called

Trackmania-Gym [58] by three main contributions: headless, container-capable training, vectorized, multi-track training, and continuous actions for TMNF. Furthermore, we explore previously uncommon algorithmic combinations, such as SimbaV2+PPO, to enhance the performance of on-policy algorithms, which have proven more practical than off-policy algorithms, specifically for TrackMania [58].

One of the main constraints of RL for TMNF is its Windows-bound nature, which makes training on a Cluster or Server, which are typically Linux-Based, difficult. This motivated us to create a headless version of the environment, which was denied to run on the BW-Uni-Cluster, sadly. We focused our efforts on continuous agents and multi-track training, proving our method of generating continuous action works, and we show the struggles of a vectorized agent.

II. PRELIMINARIES

A. Trackmania Nations Forever : A systems perspective

TMNF is a Windows-native application built upon the Microsoft DirectX 9 API. Because the training pipeline for this study targets Linux-based High-Performance Computing (HPC) clusters, the environment cannot be executed natively. Running the game on Linux requires a compatibility layer, such as Wine, to translate DirectX calls into Linux-compatible graphics APIs like OpenGL or Vulkan. Standard Wine implementations typically utilize WineD3D to translate DirectX 11 and earlier calls into OpenGL, while DirectX 12 and later calls are translated into Vulkan. An alternative optimization involves the use of DXVK (DirectX-over-Vulkan), a translation layer that provides a Vulkan-based backend for Direct3D 9, 10, and 11. While the default WineD3D/OpenGL pipeline is the standard approach, existing RL research on TMNF [1] suggests that leveraging DXVK can potentially yield higher frame rates.

B. Trackmania-Gym

Trackmania-Gym [58] is a Reinforcement Learning framework that offers gymnasium [62] compatible environments to train agents using standard RL-libraries, like Stable-Baselines3 [50]. We chose this framework to expand upon due to its flexibility and well-maintained codebase. Trackmania-Gym uses TMInterface (TMI) [20], which exposes a Python client, to communicate with TMNF directly. TMI allows Trackmania-Gym to treat TMNF as a steppable simulator, enabling the

¹"I Trained an AI for 2 Years on Trackmania. It's Breaking Records." (Accessed 12.07.2025).

manipulation of the engine's internal tick rate and simulation speed.

C. Reinforcement Learning

RL is a subfield of machine learning in which an agent learns to make sequential decisions by interacting with an environment to maximize cumulative rewards [61]. These problems are typically formalized as a Markov Decision Process (MDP) defined by the tuple $\langle S, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ [49], with S and $\mathcal{A}$ being the state and action spaces, $\mathcal{R}(s, a)$ the immediate reward received after taking action a in state s , $\mathcal{P}(s'|s, a)$ the probability of transitioning to state s' from state s after action a and $\gamma \in [0, 1)$ the discount factor weighting future rewards.

The agent interacts with the MDP by following a policy $\pi : S \rightarrow \mathcal{A}$. The objective is to learn an optimal policy π^* that maximizes the expected discounted return $J(\pi)$:

$$J(\pi) = \mathbb{E}_{\pi} [G_t], \quad \text{with } G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}. \quad (1)$$

In this formulation G_t represents the cumulative discounted reward starting from timestep t , and the expectation is taken over the trajectories induced by π .

1) *Action Spaces*: Action spaces in Reinforcement Learning are categorized as either discrete or continuous [43]. In a discrete action space, the agent selects from a finite set of k actions, $\mathcal{A} = \{a_1, \dots, a_k\}$. In contrast, continuous action spaces allow the agent to output a vector of real-valued numbers within a specified range, $\mathcal{A} \subset \mathbb{R}^k$, typically normalized to $[-1, 1]^k$ [63].

2) *Value Functions*: To find an optimal policy, RL algorithms often rely on value functions, which estimate the "goodness" of states and actions [61]. The *state-value function* $V^{\pi}(s)$ is the expected discounted return starting from state s and following the policy π

$$V^{\pi}(s) = \mathbb{E}_{\pi} [G_t | s_t = s]. \quad (2)$$

The *action-value function* (or Q-value function) $Q^{\pi}(s, a)$, represents the expected discounted return from taking action a in state s and then following policy π

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} [G_t | s_t = s, a_t = a]. \quad (3)$$

The *advantage function* $A^{\pi}(s, a)$ measures how much better taking action a in state s is compared to following the policy $\pi(\cdot | s)$

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s). \quad (4)$$

The optimal policy π^* is the one that maximizes these corresponding value functions. For this reason, most RL algorithms aim to optimize these value functions with respect to the policy in order to recover π^* [61].

D. Reinforcement Learning Paradigms

The concepts of value functions give rise to different approaches to solving RL problems. While RL very generally can be classified into Model-Free RL and Model-Based RL [61]. IN Model-Based RL, where the agent is either given the

model or tries to learn the model of its environment [33]. We focus on Model-free RL. Furthermore, RL algorithms can be classified into three categories: value-based, policy-based, and actor-critic methods [3]. The primary distinction among these paradigms lies in their learning objectives, whether the focus is on estimating value functions, directly optimizing the policy, or integrating both strategies. In the following, we briefly introduce each paradigm to provide context for the methods discussed later in this work.

1) *On-policy and Off-policy*: Specific algorithms can be classified into on-policy and off-policy algorithms. On-policy algorithms use the same policy to generate actions and collect data, while off-policy algorithms use a behavior policy to collect data in order to optimize a target policy [26, 13].

2) *Value-based Methods*: Value-based methods seek to find the optimal policy π^* by estimating the optimal action-value function $Q^*(s, a)$. This can be done either by learning it directly or indirectly through the estimation of the state-value function $V(s)$ and/or the advantage function $A(s, a)$ [65]. Once Q^* is known, the optimal policy π^* is implicitly defined by $\pi^*(s) = \arg \max_a Q^*(s, a)$. The foundation of value-based methods lies in the Bellman optimality equations [61] which express the value of a state (or state-action pair) in terms of the expected return from its successor states. For the Q -function the Bellman Optimality condition is defined as

$$Q^*(s, a) = \mathbb{E}_{s' \sim P} \left[\mathcal{R}(s, a) + \gamma \max_{a'} Q^*(s', a') \right]. \quad (5)$$

A classic algorithm in this category is *Q-learning* [66], which iteratively updates an estimate Q_{θ} of the optimal action-value function by applying the Bellman update. In modern applications, $Q_{\theta}(s, a)$ is typically approximated using a parameterized function, such as a deep neural network. This was popularized by the Deep Q-Network (DQN) algorithm [44], which is trained by minimizing the squared temporal difference (TD) error between the predicted and target Q -values, defined as

$$\delta_t^2 = \left(R_t + \gamma \max_{a'} Q_{\theta}(s', a') - Q_{\theta}(s_t, a_t) \right)^2. \quad (6)$$

3) *Policy-based Methods*: In contrast to value-based methods, which implicitly derive a policy by learning value functions, policy-based methods directly learn a parameterized policy $\pi_{\theta}(a|s)$ (e.g., a neural network), which is optimized to maximize the expected return [61]. The REINFORCE algorithm [67] established that policies can be optimized directly by following the gradient of the expected return

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a|s) G_t]. \quad (7)$$

This formulation allows estimating the policy gradient from sampled trajectories, but it often exhibits high variance due to the use of raw noisy returns G_t . Rather than directly optimizing the expected return traditional policy gradient methods have been shown to benefit from optimizing the advantage function $A(s, a)$ instead [53]. This approach leads to

an unbiased estimate of the policy gradient while significantly reducing its variance

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{(s,a) \sim \pi_{\text{old}}} \left[\frac{\pi_{\theta}(a|s)}{\pi_{\text{old}}(a|s)} \nabla_{\theta} \log \pi_{\theta}(a|s) A^{\pi_{\text{old}}}(s,a) \right]. \quad (8)$$

4) *Actor-Critic Methods*: Actor-Critic methods combine the strengths of value-based and policy-based RL [34]. They maintain two separate models: the *actor*, which represents the policy π_{θ} and is responsible for selecting actions, and the *critic*, which estimates a value function to evaluate the actor's performance. The critic provides a learning signal that helps reduce the variance of the policy gradient estimate, leading to more stable and efficient updates [60]. The actor uses this feedback to adjust the policy parameters in a direction that improves expected return.

a) *Proximal Policy Optimization*: Proximal Policy Optimization (PPO) [54] is an on-policy actor-critic algorithm designed for both continuous and discrete action spaces. It aims to improve training stability by restricting how much the policy changes between updates, following the concept of trust region methods [55] where the idea is to prevent large policy updates that otherwise can lead to unstable learning or performance collapse. Instead of enforcing an explicit trust region constraint, PPO uses a clipped surrogate objective to train the policy network

$$\mathcal{L}_t^{\text{CLIP}}(\theta) = \min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \quad (9)$$

with $\hat{A}_t = A^{\pi_{\theta_{\text{old}}}}(s_t, a_t)$ and $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$

The critic V_{ψ} is optimized by minimizing the Mean Squared Error between the predicted state value and the target return $\hat{R}_t$

$$\mathcal{L}_t^{VF}(\psi) = (V_{\psi}(s_t) - \hat{R}_t)^2. \quad (10)$$

To ensure sufficient exploration and prevent too early convergence to sub-optimal policies, an entropy bonus $S[\pi_{\theta}](s_t)$ is incorporated into the optimization. The complete objective maximized during each PPO update is a weighted combination of these three components:

$$\mathcal{L}(\theta, \psi) = \mathbb{E}_t [\mathcal{L}_t^{\text{CLIP}}(\theta) - c_1 \mathcal{L}_t^{VF}(\psi) + c_2 S[\pi_{\theta}](s_t)], \quad (11)$$

where c_1 and c_2 are hyperparameters controlling the influence of the value function error and the entropy bonus, respectively.

5) *Distributional Reinforcement Learning*: *Distributional reinforcement learning* (DRL) models the return as a random variable and aims to learn its full distribution $Z(s,a)$ for each state-action pair [5]. This contrasts with traditional reinforcement learning methods, which focus solely on estimating the expected return, i.e., the mean of the return distribution $Q(s,a) = \mathbb{E}[Z(s,a)]$. By learning the full distribution of returns DRL still allows for standard action selection based on the expected value, but the expectation is now computed over a modeled distribution which can provide a more robust and uncertainty-aware estimate [14, 15].

E. On Scaling Model Size in Reinforcement Learning

Inspired by the success of deep learning, where increasing model capacity often leads to improved performance [18, 21, 16], a natural question in RL is whether similar gains can be obtained by scaling existing methods rather than introducing new algorithmic components. Increasing the number of parameters enhances the expressiveness of a model, allowing it to represent a broader class of functions and potentially capture solutions that smaller networks fail to learn.

At the same time, higher capacity is typically associated with an increased risk of overfitting [25]. Larger networks can, in principle, fit noisy patterns in the data, which may harm generalization. However studies have shown that overparameterized neural networks often converge to relatively simple functions that generalize [69, 46].

This behavior is commonly attributed to an implicit simplicity bias, whereby standard optimization procedures and architectural choices steer highly expressive models toward solutions that are relatively simple and generalize well [56, 6].

While this phenomenon is well documented in supervised learning, where the training data distribution is fixed, its implications for RL are less straightforward. In RL, the data distribution depends on the agent's policy and therefore changes throughout training [61]. This non-stationarity introduces additional challenges, as large models may overfit to earlier experience [47], adapt more slowly to new data, and lead to unstable optimization dynamics [19, 41]. Consequently scaling model size in reinforcement learning does not necessarily yield the same consistent improvements observed in supervised learning.

1) *SimbaV2*: To address the challenges of scaling deep RL, the Simba (Simplicity Bias) architecture [39] demonstrated that injecting a simplicity bias into the network design stabilizes the training of overparameterized agents. Simba achieved this by utilizing observation normalization, residual feedforward blocks, and layer normalization, which collectively allowed RL models to scale in parameter count without suffering from severe overfitting.

SimbaV2 [38] extends Simba by focusing more directly on the optimization instabilities that limit scaling in RL. A central issue is uncontrolled norm growth during training, which affects features, parameters, and gradients. Feature norms can grow due to biases in the TD loss, causing certain dimensions to dominate and leading to overfitting and reduced plasticity [41, 36]. At the same time, parameter norms may increase without bound, effectively reducing the learning rate and slowing down updates [19]. Gradient norms are also unstable, as they depend on reward scale and can be affected by outliers. These effects become more pronounced with larger models making scaling in RL more challenging than in supervised learning.

SimbaV2 addresses these issues by combining several stabilization techniques within a single framework. It introduces three key modifications to the Simba architecture:

- **Hyperspherical Feature Normalization**: Replaces all

layer normalizations with ℓ_2 -normalizations, projecting feature vectors onto the unit hypersphere.

- **Hyperspherical Weight Normalization:** After each gradient update, weights are ℓ_2 -normalized eliminating the need for weight decay.
- **Distributional Value Estimation with Reward Scaling:** Q-value functions are learned using distributional RL, with reward scaling applied to maintain unit variance in the targets throughout training.

For a comprehensive description of the SimbaV2 architecture and its implementation details, we refer the reader to the original work [38].

F. Framing Regression as Classification

Modern deep learning architectures have achieved remarkable success across a wide range of tasks, many of which are formulated as classification problems [9, 68]. Motivated by this observation, recent work suggests that even when a task is naturally posed as a regression problem, reformulating it as a classification problem can lead to improved performance [59]. One explanation for this phenomenon lies in the optimization properties of the respective loss functions. Classification objectives such as cross-entropy often provide more stable and informative gradients compared to the squared loss used in regression [30, 31]. Moreover, they induce a different implicit bias in gradient descent, encouraging the learning of feature representations that are more robust and generalize better to unseen data [59]. This perspective has also been explored in the context of RL. In particular, recent work demonstrates that replacing regression-based value function learning with a classification-based approach can significantly improve both performance and scalability [22]. A key technique in this line of work is the use of the Histogram-Gauss loss [30] which we will discuss in more detail in the following subsection.

1) *Histogram-Gauss Loss:* Standard regression models are typically trained by minimizing the mean squared error (MSE), for which the optimal predictor is the expectation $f_\theta(x) \approx \mathbb{E}[Y|x]$ [8]. While theoretically well-founded, this limits the model to predicting a single point estimate. As a result, information about the shape of the conditional distribution such as uncertainty or multimodality is not captured. To address these limitations the work [30] propose a distributional regression framework that learns the full conditional distribution $p(y|x)$ by discretizing the target space.

In this framework, a continuous target $y \in [a, b]$ is partitioned into k uniform bins of width w . The network f_θ is trained to output a categorical distribution $f(x) \in [0, 1]^k$ over these bins. The learning objective is to minimize the KL-divergence between a predefined target distribution $p(y)$ and the predicted density q_x which they define as $f_i(x)/w$ per bin. This simplifies to minimizing the cross-entropy

$$\mathcal{L} = - \sum_{i=1}^k p_i \log f_i(x), \quad (12)$$

where p_i represents the probability mass of the target distribution within the i -th bin $[l_i, l_{i+w}]$. Because this approach

requires a Cumulative Distribution Function (CDF) that can be evaluated, the framework focuses on probability distributions for which the CDF is available in closed form.

For the HL-Gauss variant, the target distribution $p(y)$ is defined as a truncated Gaussian centered at the ground-truth label y with a fixed variance σ^2 which allows the target probabilities p_i to be calculated in closed form via the error function.

III. MDP FORMULATION

In this section, we want to introduce the MDP (S, A, R, ρ, s_0) that models the reinforcement learning environment. In particular, this section introduces the actions A , Observations S , and reward function $r_t = R(s_t, a_t)$.

A. Actions

Trackmania-Gym heavily investigated discrete actions. This work implements the capability to also train with continuous actions, which is where we will focus. An actionspace can therefore be defined discrete, as well as continuous.

1) *Discrete Actionspace:* The Discrete Actionspace reflects a similar input as available to humans, which consists of only the four inputs "gas", "brake", "left", and "right" in each environment step. Combining these four into every possible combination gives us 12 discrete actions. Each action is represented as $a \in \{0, 1\}^4$, a four-dimensional Boolean vector, which is 1 if the corresponding action is chosen. The action reduction from 16 to 12 actions is achieved by ignoring canceling actions, e.g., ("left" = true, "right" = true) [58].

2) *Continuous Actionspace:* Our final action space for a continuous agent is a 3-dimensional action space $A = [-1, 1]^3$ for steering, accelerating, and braking. Especially in Trackmania, it is important for the agent to be able to break and accelerate at the same time, which makes a two-dimensional action space for acceleration-control and steering impractical. For the acceleration and braking dimensions, the minimal action-value -1 is interpreted as 'do nothing', whereas for the steering dimension, the minimum and maximum correspond to the maximum values of Left and Right, respectively, and 0 corresponds to 'do nothing'.

B. Observations

All of our tested observations consist of a state-vector $s \in \mathbb{R}^d$, containing vector-like information and an image $\in \mathbb{R}^{w \times h}$. We did not add noise to any of our observations, as in our case, the simulator (TMNF) is also the real-world application of the agent.

1) *Image-Observations:* Our image observation is a 128×128 grayscale image, as displayed in Figure 1.

2) *Vector-Like Observations:* The state vector contains information directly extracted from the game, as well as the next $k(= 10)$ reference-line points in the car coordinate system. A complete list of observations along with their dimensionality is provided in Table VI.



Fig. 1: Example 128×128 grayscale example observation during training. We removed all graphical display elements, e.g., the time-driven or the speed.

3) *Normalization*: In order to normalize the images and state vector, we divided the image element-wise by 255 to obtain a normalized image. For the state vector, we defined maximum values for each type in the observation vector and divided the observation given by the observation manager by that value, which are denoted in Table VI.

C. Rewards

We designed the reward function of the agent as a weighted combination of multiple components

$$r_t = r_t^a + \lambda^w r_t^w + \lambda^g r_t^g + \lambda^s r_t^s + \lambda^p r_t^p. \quad (13)$$

Compared to other projects, we significantly reduced our reward function, leaving out any rewards tailored by prior knowledge about Trackmania, and also intermediate rewards that do not optimize the long-term goal of the agent. The complete reward with weights is denoted in Table VII.

1) *Accumulated Distance*: The accumulated distance reward measures the progress of the car along the center line by adding the distance of all points passed, normalized by the total distance D . The euclidean distance between two points $p_i, p_j \in \mathbb{R}^3$ is denoted $d(p_i, p_j)$ and the set of points passed in one environment step is denoted by the set $P_t = \{p_i, p_{i+1}, \dots, p_k\}$, resulting in the final accumulated distance

$$d_t^a = \frac{1}{D} \sum_{p_i, p_j \in P_t} d(p_i, p_j), \quad j = i + 1$$

In order to emphasise speed to the agent, we scale this reward by multiplying covered distance by the number of points the agent passed $n_p = |P_t|$, raised to a small exponential factor $\eta = 1.2$ to get the final accumulated distance reward

$$r_t^a = d_t^a \cdot (n_p)^\eta.$$

2) *Constant Rewards and Punishments*: Constant rewards are fixed values given to the agent either at every environment step or during specific events. These events include completing the race r_t^g , getting stuck (being unable to move) r_t^s .

Many projects use a Lateral-Distance Reward in early stages of training so the agent learns to drive in the center of the track. We found that this intermediate behaviour is not necessary for the agent to succeed and is indirectly achieved, by giving a constant punishment r_t^w for being in contact with a wall.

D. Termination

An episode terminates if the number of environment steps within that episode exceeds a previously fixed timeout, which is determined by timing a human driving the track and multiplying the number of environment-steps by three. Furthermore, an episode terminates if one of the other termination conditions, which are: the car is stuck, or the agent finishes the race. In all of these scenarios mentioned above, except for a regular timeout, the agent is dealt a reward.

IV. METHOD

In this section, we present our extensions to the Trackmania-Gym framework, focusing on the development of a headless cluster implementation, action space continuization, and multi-track training capabilities.

Building upon the foundations established by the previous Trackmania-Gym research, our work focuses primarily on model-free RL algorithms. While model-based RL algorithms theoretically offer superior sample efficiency by leveraging learned environment models, they remain notoriously difficult to train due to compounding model errors and instability in long-term predictions [12, 64, 17]. In contrast, since TMNF is as a fast and easily resettable simulation environment, model-free methods are often more suitable due to their relative simplicity and robust empirical performance [28]. We therefore selected PPO as our baseline for its stability in on-policy learning [54]. Furthermore, we investigate the integration of the SimbaV2 framework into the on-policy case. Although SimbaV2 was primarily designed for and applied to off-policy algorithms to improve scaling and stability, we explore how these methods can be extended to on-policy methods.

A. Running TMNF on a HPC

The use of a HPC infrastructure was mainly motivated by the computational cost of modern deep RL methods. Training agents in complex environments such as TMNF requires extensive hyperparameter tuning, including systematic variation of learning rates, discount factors, and network architectures. Furthermore RL is known for its high variance, making it necessary to run multiple training instances with different random seeds in order to obtain reliable results [10]. In addition large neural networks introduce a substantial computational load due to repeated forward and backward passes during training. Running these experiments on HPC systems allows such workloads to be distributed across powerful compute nodes, which significantly reduces overall training time and makes a wider exploration of the parameter space possible.

1) *Containerization*: Deploying TMNF on the target cluster required a container-based setup due to incompatibilities with the cluster environment. The cluster system is Linux-based and provides only a limited set of pre-installed software, in particular required components such as Wine and related dependencies are not available and cannot be installed. Moreover, TMNF is not a simple executable but depends on a collection of binaries, graphics libraries, and system-level components. These factors render a standard file-based deployment technically infeasible.

To address this, we used a containerized approach that bundles all required dependencies, including the necessary runtime environment and graphics support, into a single image.

2) *Implementation*: The container image is based on an Ubuntu runtime with NVIDIA CUDA ² and OpenGL ³ support. This base setup includes the Wine compatibility layer required to run the TMNF binaries. Since the compute nodes on the cluster do not provide physical displays, a headless rendering setup was required. For this purpose, Xvfb (X Virtual Framebuffer) is used to emulate a display in memory. In addition, a lightweight window manager is included to handle the graphical interface, which allows the framework to query window properties as required by TMI.

On top of this, we provide a second image that uses the Vulkan API instead of OpenGL. This variant installs the necessary Vulkan libraries and integrates DXVK, enabling Direct3D calls to be translated to Vulkan rather than OpenGL. As Xvfb has limited support for Vulkan-based rendering [4], this setup relies on a customized headless Xorg configuration to provide the virtual display.

Both of these setups support hardware acceleration and enable direct use of the host GPUs from within the container.

3) *Deployment Strategy*: The software stack was initially built using Docker ⁴. However most clusters do not support Docker due to its reliance on root-level privileges. To comply with the security restrictions we instead used Apptainer (formerly Singularity) and Enroot⁵, which are designed for rootless container execution in HPC environments.

To ensure consistency across platforms, we implemented Apptainer definition files that mirror the multi-layered structure of our original Docker images. We also leveraged the native conversion utilities provided by these runtimes to port existing Docker images into HPC-compatible formats without loss of configuration.

B. Integration of SimbaV2 into PPO

While the original Simba architecture demonstrated performance gains when applied to PPO [39], the subsequent SimbaV2 framework focused exclusively on off-policy algorithms [38]. In this work, we propose SimbaV2-PPO, a novel integration of the SimbaV2 architecture into the on-policy PPO framework. We argue that the core stability mechanisms of

SimbaV2 specifically its hyperspherical feature and weight regularisation are algorithm-agnostic and should provide similar scaling benefits to PPO. Our implementation incorporates hyperspherical normalization, observation and reward scaling, and a distributional critic.

1) *Hyperspherical Normalization*: We utilize the full SimbaV2 residual block architecture, which replaces standard Layer Normalization and weight decay with explicit ℓ_2 -projections of features and weights onto the unit hypersphere. This structural constraint prevents the uncontrolled norm growth typically observed in deep reinforcement learning.

2) *Normalization*: Following the SimbaV2 framework, we apply running statistics to normalize each dimension of the input observation o_t to zero mean and unit variance. This ensures that the input features remain within a consistent numerical range, facilitating stable optimization from the first layer. Furthermore we adopt the same adaptive reward scaling and thresholding mechanisms as in SimbaV2 to stabilize gradients across varying return magnitudes. This process ensures that target returns remain strictly within a predefined support $[G_{\min}, G_{\max}]$, preventing gradient explosions caused by high-variance rewards.

3) *Distributional Critic*: To bridge the gap between SimbaV2's off-policy design and the on-policy requirements of PPO, we replace the standard MSE loss (10) with the HL-Gauss loss (12). We define the target space for the categorical distribution to match the $[G_{\min}, G_{\max}]$ range established by the reward scaling. By reframing value regression as a categorical classification task, we satisfy the SimbaV2 requirement for bounded gradient norms while maintaining the standard actor-critic framework.

C. Vectorization

Trackmania-Gym offers strong single-environment support, but lacks support for vectorized training. We expanded the framework's capability to support vectorized trainings by chaining multiple single-agent environments together and collecting rollouts in batches of, as illustrated in Figure 2. This exposes an already vectorized interface to our learner process, which proved to be much more stable than native vectorization methods as implemented in stable-baselines3.

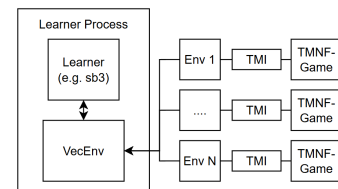


Fig. 2: Overview of the communication flow when training with the vectorized environment. Each environment instance works completely unaware that it is part of a Vectorization. Each environment also has its own Game-Instance.

1) *Multi-Track Training*: Our vectorization implementation not only allows us to collect rollouts in batches, which significantly improves training times for single track trainings, but also allows us to train on multiple tracks at the same time.

²<https://developer.nvidia.com/cuda-toolkit>

³<https://www.opengl.org/>

⁴<https://www.docker.com>

⁵<https://github.com/NVIDIA/enroot>

2) *Normalization*: Training in a vectorized environment leaves two options for the normalization of rewards and observations. The first one is just the normalization per environment, as implemented in the single-environment case, which means to normalize accross a single rollout, i.e.

$$\hat{x}_i = \frac{x_t - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

where μ, σ^2 are the mean and variance of either the observation or reward. Secondly, it is also possible to normalize observations and rewards across the vectorized batch with

$$\hat{x}_{i,e} = \text{clip} \left(\frac{x_{i,e} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, -c, c \right)$$

where μ_B and σ_B are the running statistics of the mean batch and standard deviation. We also clip all values to $[0, 1]$ to prevent any outliers. To track statistics in both cases, we used Stable-Baselines3 [50] implementation of a running mean.

D. Continuous Actions

In general, optimizing a policy in a discrete action space is more complex than in the discrete case, but these policies often yield better results than discrete policies once converged [32]. Furthermore, some state of the art reinforcement learning algorithms like SAC [27] require continuous actions. However, TMNF is inherently implemented with binary action commands for acceleration and braking, i.e., either 0% or 100% acceleration or braking at any single step in time. In order to still be able to send continuous action commands, we achieved continuity in the actions by interpreting a continuous action as a series of discrete actions. Our environment-simulator setup has two main stepping metrics: the simulation steps and environment steps. Our environment steps are set to n_{aps} , the number of actions per second, while the simulator does 100 steps every in-game second. This means the number of simulation steps between environment steps equates to

$$n_{sbe} = \frac{100}{n_{aps}}.$$

This equates to the number of continuous buckets that we can use. For each simulation step between environment steps, we choose how many of them execute a braking or acceleration action

$$f : [0, 1] \rightarrow \{0, \dots, n_{sbe}\}, \quad f(a) = \lfloor a \cdot \frac{100}{n_{aps}} \rfloor = \lfloor a \cdot n_{sbe} \rfloor$$

as visualized in Figure 13. Because both continuous actions break and gas, we can just interpret the continuous value as a rounded fraction of how many buckets are either braking or accelerating, as shown in Figure 3.

1) *Limitations*: Because the number of buckets k is limited by the number of simulation steps between environment steps, and therefore directly set by the actions per second, we are unable to increase the continuization accuracy. Additionally, decreasing the actions per second parameter caused frequent communication failures between TMNF and our framework,

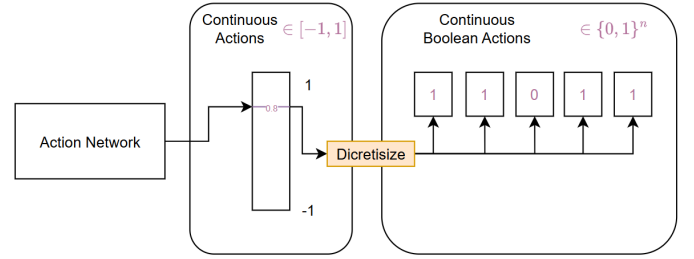


Fig. 3: Schematic visualization of the discretization of a discrete action $a = 0.8$ into five discrete buckets.

which requires a restart of the training setup, therefore increasing the overall wall-clock time significantly. Unfortunately, we were unable to identify the root cause and therefore decided to limit actions per second to 20. This also means the agent only has five continuization buckets.

2) *Actionspace Squashing*: Our action space is defined by $[-1, 1]^3$, naturally, we use a tanh squashing function in the policy network to be able to interpret the actions by the policy directly. Due to boundary effects, however, it is difficult for the agent to perfectly use the whole action space, i.e., actions at the upper or lower range of the action space are rarely used, due to the Gaussian nature of the policy [11]. We therefore clip the agents' actions internally to $[-0.7, 0.7]$ and squash the action bins to that interval

$$a \rightarrow \begin{cases} 1.0, & a > 0.7 \\ 0.7a & a \in [-0.7, 0.7] \\ -1.0, & a < -0.7 \end{cases}$$

which ensures our agent is able to reach the maximum and minimum action.

3) *Environmental Noise Investigation*: We thoroughly investigated our training setup to exclude any unwanted side effects as best as possible. We found that if the learner process takes too long to send a command to the TMI, e.g., because a policy update is taking too long, then the game deactivates the mode as a steppable simulator and resumes regular execution. The state the car was in at the last communication s_1 is applied, which means the car moves. This creates a discontinuity from the last state s_1 the agent was in to the state after communication resumption s_2 . Although we are able to modify our rewards to detect that discontinuity, which makes this step for the agent invalid, we were unable to completely remove this discontinuity. Given that, for on-policy methods, this happens at most once every 2048 steps, we attribute this discontinuity to general environment noise.

V. EXPERIMENTS

This section details the experiments and results. We investigate how training across multiple tracks simultaneously enhances an agent's ability to generalize, evaluate a curriculum-based approach inspired by *tmnf gym* that utilizes dense reward pre-training followed by sparse reward fine-tuning. Additionally, we conduct a analysis between the SimbaV2-PPO architecture and a standard PPO baseline to infer the

benefits of the SimbaV2 modifications. Finally we investigate the computational efficiency gains achieved by migrating the training process from local hardware to a high-performance computing cluster.

A. Experimental Setup

1) *Tracks*: To ensure reproducibility and establish a fair comparison with existing benchmarks, we employ a diverse set of racing environments. This selection includes five official TMNF tracks ranging from introductory layouts to more complex designs as shown in Figure 10. To further evaluate the agent’s generalization capabilities, we also developed a suite of custom maps (e.g., Simple1, Curvy, and Another1–3), illustrated in Figure 11. Together, these environments provide a diverse and challenging framework for evaluating driving performance. Additionally, to account for varying track complexities, each map is assigned a specific episode length limit meaning the maximum number of steps the agent is allowed to take to finish the race, which is detailed in Table I.

2) *Observation Space and Reward Structure*: All experiments utilize the multi-modal observation space detailed in Section III-B. The reward function follows the formulation introduced in Section III-B, with individual reward term weights provided in Table VII unless specified otherwise. Notably for observation processing, the baseline PPO agent relies on the manually handcrafted state normalization factors outlined in Table VI. In contrast the SimbaV2-PPO agent does not use this manual normalization, relying instead on running mean normalization for each observation.

3) *Network Architecture and Feature Extraction*: Because the observation space combines image data and state vectors, we adopt the feature extraction approach introduced by TMNF-Gym. As illustrated in Figure 4, the network utilizes separate feature encoders for each modality: a Vision Encoder for the images and a 2-layer MLP State Encoder for the physical parameters.

The latent representations from both encoders are concatenated and passed through a shared MLP. This vector serves as the latent representation of the entire observation space and is fed into the Actor and Critic networks.

For the SimbaV2-PPO agent we adopt the core architecture proposed in the original framework [38], with layer specifications summarized in Table III. Importantly to maintain architectural consistency in the SimbaV2 implementation, all feature extractors including the vision encoder, the state encoder, and the shared MLP strictly adhere to the unit-normalization constraints. Detailed specifications of the vision encoders used for both algorithms are provided in Table V and Table IV.

4) *Training and Hyperparameters*: Our analysis investigates the performance of standard PPO and our SimbaV2-PPO implementation. To isolate the specific impact of the SimbaV2 architecture, all shared algorithmic hyperparameters are kept strictly identical across both algorithms. Table II summarizes these shared parameters alongside the parameters exclusive to

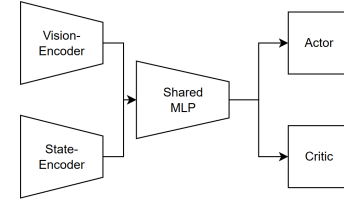


Fig. 4: Shared Feature-Encoder Pipeline. The latent representation of Vision- Encoder and State-Encoder are concatenated, which is then given to the shared-MLP, which creates a latent representation of the whole observation vector.

the SimbaV2-PPO agent. Furthermore, both agents utilize an identical linear learning rate decay schedule, as visualized in Figure 12.

B. Practical Constraints in Cluster Deployment

Despite the theoretical advantages of the distributed architecture, the transition from local development to cluster deployment encountered significant problems that ultimately prevented its use for large-scale training. While the environment functioned as intended under Docker on local workstations, the migration to the cluster’s supported runtimes introduced significant technical issues.

1) *Incomplete 32-Bit Driver Injection*: As a 32-bit legacy application, TMNF requires specific 32-bit host-side graphics drivers for hardware acceleration. Unlike Docker, the cluster’s Apptainer runtime does not automatically map these 32-bit libraries [2]. Manual binding attempts resulted in versioning conflicts with the 64-bit stack required by the deep learning framework, resulting in unstable environment. We therefore migrated to Enroot, which provides more robust driver injection. However as detailed below, its utilization was ultimately constrained by broader administrative and kernel-level restrictions.

2) *Vulkan Hardware Rendering Incompatibilities*: The compute nodes of the cluster are equipped with data center GPUs (e.g., NVIDIA H100), which are primarily designed for parallel compute workloads rather than 3D graphics rendering. Testing confirmed that these data-center GPUs do not support the Vulkan extensions required when running TMNF via DXVK. While hardware-accelerated OpenGL was successfully validated using simple benchmarks (e.g., glxspheres64), we were unable to test this rendering path for the game engine due to the following system-level restrictions.

3) *Kernel-Level Security Policies*: The final and most definitive blocker was the cluster’s kernel security configuration regarding memory allocation. To execute Windows-native binaries, Wine requires low-address memory mappings at specific page addresses. However, the cluster enforces a strict threshold to mitigate security vulnerabilities. Cluster administrators confirmed that this policy is non-negotiable and cannot be lowered. Consequently, the application encountered an immediate segmentation fault during startup. This failure persisted even when using software rendering, as the memory mapping requirement is a fundamental constraint of the Wine

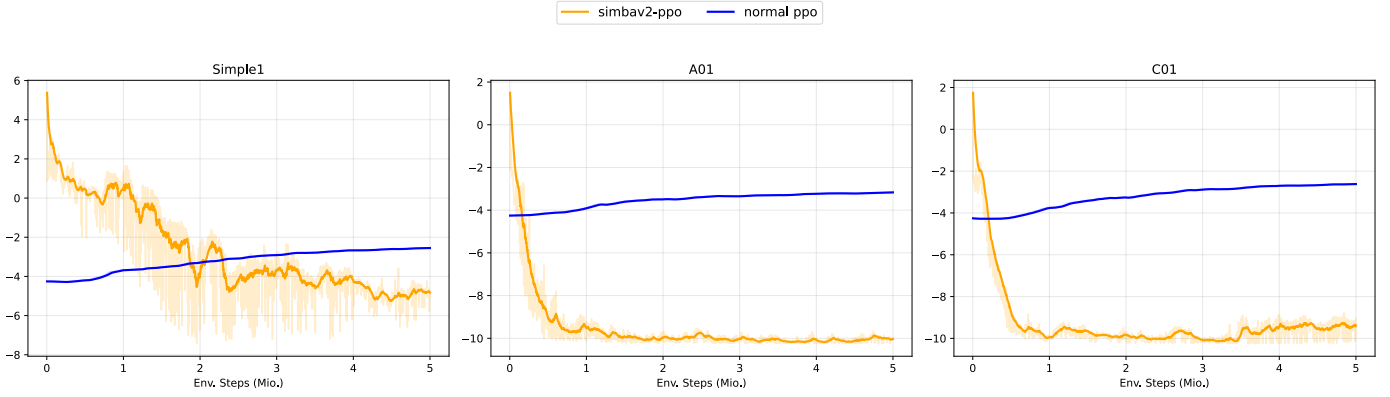


Fig. 5: **Entropy loss across evaluation tracks.** Entropy loss curves for the Simple1, A01, and C01 environments. The y-axis shows the *entropy loss* (negative policy entropy), so decreasing values correspond to increasing entropy.

layer itself.

Although the container framework functioned correctly in local tests, the technical challenges described above forced us to abandon the cluster-based approach. Consequently, the results presented in the following sections were derived from local Windows environments. While this allowed us to bypass the containerization and compatibility issues entirely, it came at the cost of significantly lower throughput compared to the intended cluster deployment. Due to this reduced computational capacity and the high overhead of the deep architectures we intend to use, our preliminary evaluations were restricted to a single seed and a single training run per track.

C. Comparing SimbaV2-PPO against normal PPO

We evaluate the performance of the proposed SimbaV2-PPO framework against a standard PPO baseline to isolate the benefits of the SimbaV2 architectural components. The comparison is conducted across three tracks of varying complexity: Simple1, A01, and C01. To ensure a fair comparison, the baseline PPO utilizes the same feature encoders as SimbaV2-PPO and the overall network architecture was adjusted to match the dimensions specified in Table III. The only difference between the network architectures is the ℓ_2 normalization within SimbaV2-PPO which is not being used by the baseline implementation. In addition each evaluation point is derived from five independent episodes. We report the Success Rate, which is the percentage of evaluation episodes in which the agent successfully crosses the finish line, the Steps Taken, which is, given the agent reached the finish line, the average number of timesteps required to reach the goal, and the return. The results between SimbaV2-PPO and the standard PPO baseline are illustrated in Figure 14. Across all three evaluation tracks the SimbaV2-PPO agent demonstrates superior performance. In all scenarios, SimbaV2-PPO converges to a high success rate significantly faster than the baseline. For example, on the *Simple1* track the SimbaV2-PPO agent achieves an 80% success rate in approximately 0.5 million environment steps, whereas the standard PPO baseline requires roughly 1.5 million steps to reach the same threshold. Similarly the episodic return increases at an accelerated rate

across almost all test cases. Furthermore, the mean steps taken to complete the tracks are consistently lower for the SimbaV2-PPO agent indicating a more rapid optimization of the Time Attack objective.

Beyond the quantitative metrics, qualitative observations of episodes reveal that the SimbaV2-PPO agent learns a significantly smoother and more stable driving policy. Its behavior closely resembles that of an experienced human player, the agent successfully learns advanced mechanics, such as initiating drifts to navigate corners efficiently, while actively avoiding collisions with the guardrails something the standard PPO agent fails to master. Overall these trends strongly suggest that the architectural integrations of SimbaV2 substantially improve the base PPO algorithm.

1) Entropy exploit: Across all evaluated tracks, we observed a consistent anomalous driving pattern during the mid-to-late stages of training for the SimbaV2-PPO agent. On straightaways, the agent executed high-frequency oscillating steering behavior, rapidly alternating between left and right inputs while maintaining a relatively straight trajectory. The root cause of this behavior becomes evident when analyzing the policy entropy in Figure 5. For all three evaluation runs, the entropy of the SimbaV2-PPO agent skyrockets compared to the baseline. Interestingly, the agent did not exhibit this erratic behavior in its gas and braking controls. This instability is a cause of our action space design and how the environment bounds inputs. Specifically, the gas and brake actions are clipped to a range of $[-0.67, 0.67]$. The agent learned a policy that outputs extreme action values (e.g., > 0.67) for acceleration. This artificially inflates the entropy bonus in the PPO objective.

Steering however is not clipped leading the policy to generate extreme steering outputs that manifest as the observed oscillating behaviour. The agent effectively learned to maximize the entropy objective by exploiting the environment's action bounds on straightaways. To validate this hypothesis, we evaluated the SimbaV2-PPO agent on the *Simple1* track using varying entropy coefficients. We reduced the standard coefficient from 1^{-3} to 1^{-4} and 0. As illustrated in Figure 6,

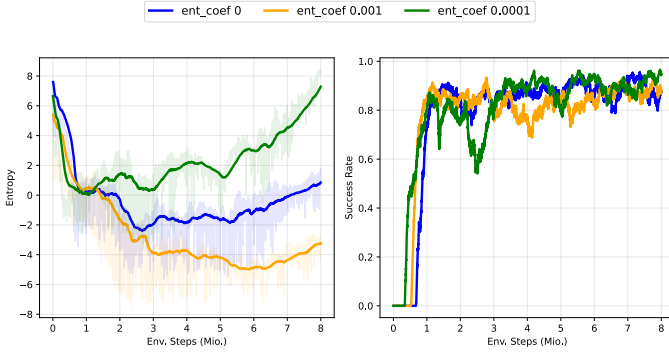


Fig. 6: **Ablation of the entropy coefficient.** Policy entropy and success rate on the Simple1 track using different entropy coefficients. Reducing the coefficient from the baseline 1^{-3} to 1^{-4} or 0.

decreasing the weight of the entropy coefficient prevents the policy from exploiting the bounds. Lower coefficients suppress the entropy explosion and dampen the oscillating behavior, while maintaining and in some phases slightly improving the overall success rate of the agent.

D. Drawbacks to Comparing off-policy to on-policy algorithms

Although it would be interesting to compare the performance of off-policy algorithms to on-policy algorithms, due to our computational limitations, this comparison is not representative. The highest Update to Data (UTD) ratio we were able to use was $\frac{1}{4}$; even then, training took almost three times the time of on-policy. Good policy algorithms like SAC set a UTD=1 as baseline [27]. This is important, as the performance of off-policy algorithms considerably increases by increasing UTD. Modern methods like Cross-Q can reach SAC performance with UTD=20 at UTD=1 [7]. Frameworks such as Bigger Regularized Optimistic (BRO) even expect UTD > 8, experiencing an almost 40% performance decrease when training with UTD=1 [29]. Because we are not able to fulfill a UTD=1, and off-policy algorithms took so much longer to train, we decided to focus on on-policy instead.

E. Sparse Reward

Sparse rewards have been proven to be quite successful in RL settings [57, 52]. Sparse Rewards has been tried in TMNF-RL setups already, however agents almost never get any reward signal, as the finish line is almost impossible to reach for the first time without prior knowledge, preventing any learning [58]. For this reason, we decided to use an agent we pretrained for a couple of iterations until the finish line can be continuously reached. Afterwards, we switched all reward signals off, except for a scaled race-finished reward

$$r_t = \frac{1}{n_{steps}}$$

where n_{steps} is the number of environment steps the agent took to reach the finish line. Unfortunately, this agent completely unlearned almost all behaviour, as can be seen in Figure 7, where the success rate drops from 1.0 down to 0.0. Although

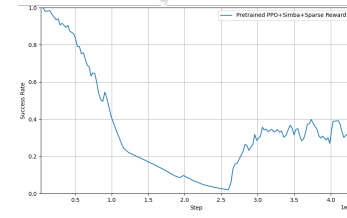


Fig. 7: **Success Rate for sparse rewards** of a pretrained SimbaV2 agent on Simple1.

the success rate climbs again later in the run, the overall entropy of the policy never recovers, as can be seen in Figure 15.

F. Multitrack Training

In our first multi-track training with standard PPO, we used per-environment normalization. This, however, yielded unusable policies, as the explained variance for this kind of normalization does not stabilize and oscillates around 0.7, as can be seen in Figure 8. This type of explained variance leads to unusable feedback from the critic, severely hindering learning overall. We also observed this in the total reward per environment, which experiences almost no improvement over 10^7 environment steps, as can be seen in Figure 16.

1) *Vecnormalize*: Normalizing across the batch of the vectorized environment significantly improved the explained variance to up to 0.96 – 0.99. However, we fear the high explained variance led to large feedback from the critic, causing extremely high clip-fractions, i.e. the percentage of PPO updates that need to be clipped as can be seen in Figure 9. A regular clip-fraction of 0.1 is considered high in PPO trainings, a clip-fraction of 0.5 is extremely wasteful, as once the clipping mechanism is activated, all gradients are zero, meaning no learning. In contrast to normalization per environment, however, all environments successfully managed to finish the tracks, as can be seen in Figure 8.

2) *SimbaV2 for Multitrack*: After the improvement from per-environment normalization to Batch-Normalization, we also want to train SimbaV2 agents for multi-track settings. Compared to per-environment normalization and Batch-Normalization, SimbaV2 agents have a explained-variance which is worse than batch-normalized agents, but better than per-environment normalized agents, as can be observed in Figure 8. For PPO agents in general, the explained variance of SimbaV2 agents is completely acceptable. Importantly, the clip-fraction of SimbaV2 agents is much lower than for VecNormalize agents, trained on the same track, as visualized in Figure 9. Agents trained with SimbaV2 are generally able to pass the finish line in all environments, as observable in Figure 8.

G. Comparison to Single-Track

The performance difference of an agent trained on one track only versus being trained on multiple tracks is significant. We compared the performance of a SimbaV2 agent trained on B03 in a Multi-Track setting with A02, B01, C01, C03 to a

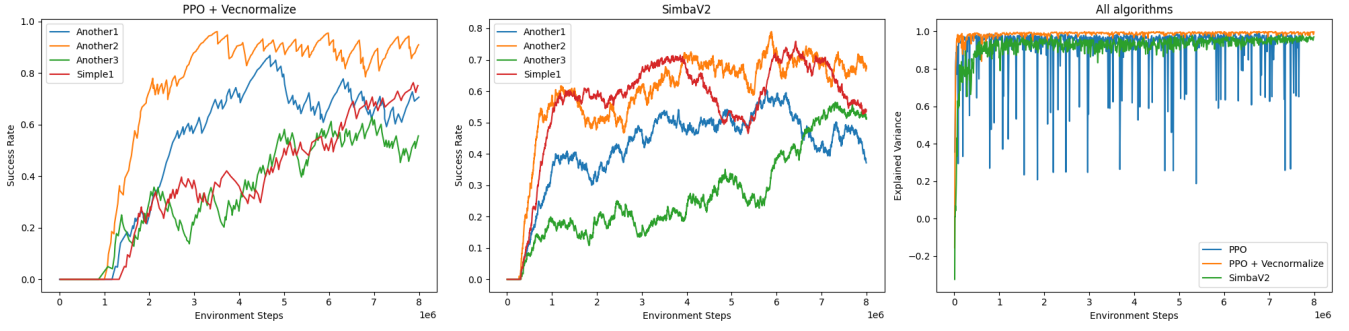


Fig. 8: **Multi-track training results.** The left and middle plots show the fraction of finished races of regular PPO agents trained with Vecnormalize and SimbaV2+PPO agents. The plot on the right compares the explained variances of PPO normalized per Environment, Vecnormalize, and SimbaV2.

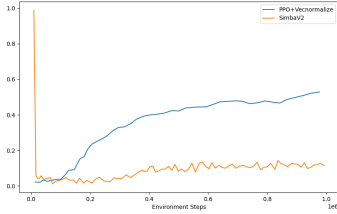


Fig. 9: **Clip-Fraction** of SimbaV2 and Vecnormalize agents in multi-track training, trained on the maps Another1, Another2, Another3, Simple1.

SimbaV2 agent that was only trained on B03. The difference in performance is visualized by the reward in Figure 19.

It is important to note that this difference is highly dependent on the other tracks in the multi-track setup, as for example when comparing a SimbaV2 agent trained on Simple1 and similar maps, which are considerably easier than A02, B01, C01, C03 and B03, the performance gap between Multi-Track agent and Single-Track agent is much lower, as can be seen in Figure 18.

H. Choosing a single track

One behaviour we observed when training in Multi-Track settings is that the agent optimizes its behaviour to be acceptable in all environments, yet stops optimizing all environments equally after that point. This can be seen in the stagnation of races finished, for almost all tracks except for one or at most two, as seen in Figure 8 and Figure 17. Our results show this is consistent for the SimbaV2 agent and the Vecnormalize agent.

VI. CONCLUSION

This student project successfully extended the Trackmania-Gym framework by introducing continuous action spaces, vectorized multi-track training, and a headless, containerized architecture. While strict system-level constraints restricted our own deployment on HPC clusters, the containerized architecture we developed serves as a foundation for other researchers, allowing those not facing the same infrastructure limitations to bypass deployment hurdles and significantly scale their computational resources.

Through a series of experiments, we validated our action continuization method, demonstrating that agents trained with continuous actions are not only usable in TMNF but in some cases slightly outperform traditional discrete-action baselines. Furthermore, our novel integration of the SimbaV2 architecture with PPO yielded a substantial performance boost, accelerating convergence and producing smoother, more stable driving policies compared to standard PPO. In the context of multi-track training, we established that batch-level normalization across the vectorized environment is strictly superior to per-environment normalization for stabilizing the learning process. While SimbaV2-PPO outperformed standard PPO in these multi-track scenarios, it could not completely resolve the environment-difficulty imbalance, agents still tended to optimize for a single track while stagnating on others. Finally, while our overall MDP formulation succeeded across a diverse set of tracks, we demonstrated that sparse reward structures in TMNF inevitably lead to training instability and policy collapse, even when fine-tuning a densely pre-trained model. We also concluded that off-policy methods, while theoretically sample-efficient, present practical drawbacks regarding training speed and strict update-to-data (UTD) requirements on limited hardware, reinforcing the practicality of our on-policy approach.

A. Future Work

To address the problems encountered during distributed training in this work, we propose a decoupled client-server architecture. In this setup, the client manages the simulation environment running multiple parallel instances of TMNF to collect data. Whereas the server (in this case, the HPC-Cluster), hosts the neural network models and performs all gradient updates. By offloading the computationally intensive learning phase to the cluster's GPU nodes while keeping the environment simulation local, this approach bypasses the previously mentioned limitations on running the full game engine on the cluster. Furthermore, we suggest investigating the use of the Validation Mode for TMI. In this mode, graphical rendering is disabled, and only the raw physics simulations are executed, significantly increasing execution speed. While this approach reduces the GPU dependency for rendering, software

compatibility with the Cluster (Wine) remains a hurdle. Besides these software-engineering-based extensions, a next step for this research is to incorporate prior knowledge into the learning process, given that TMNF provides a massive, publicly available dataset of millions of community-generated replays. We think that an Offline-to-Online (O2O) RL framework could be successful here. In the offline phase, the agent learns exclusively from a fixed dataset of experience without any interaction with the environment. Following this, the online phase begins, in which the pretrained agent is fine-tuned through direct interaction with the environment. To make the O2O Transition successful prior work suggests that policy expressivity plays a crucial role [40]. The standard Gaussian policy used in this study assumes a unimodal action distribution, which limits its ability to capture the multi-modal nature of human driving behavior. Existing O2O approaches have therefore explored more expressive policy classes such as Diffusion models to better represent this complexity [40, 48]. Following these findings, another important direction for future investigation is to look at how such expressive policy representations influence the performance of a TMNF-agent.

1) *SimbaV2-PPO*: In this work, we integrated the HL-Gauss loss [30] to fully adapt the SimbaV2 framework [38] to the PPO algorithm. As demonstrated in the original work, the HL-Gauss loss provides significant optimization benefits by being Leibniz-bounded, which ensures more stable gradient scaling [30]. However, a current limitation is that the learned distribution remains constrained to a unimodal Gaussian, which lacks the expressivity for highly complex behavior. A natural extension of this approach would be to investigate more complex distributions that still allow for a closed-form CDF. This would enable the agent to represent multi-modal behaviors while maintaining the advantages of the histogram-based loss formulation.

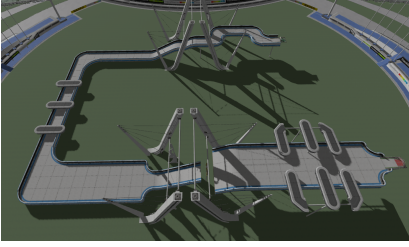
REFERENCES

- [1] Agade09 (GitHub-Username) and pb4git (GitHub-Username). *Linesight-AI Documentation*. <https://linesight-rl.github.io/linesight/build/html/>. Accessed: 17 August 2025. 2025.
- [2] ehfd (Seungmin Kim). *Provision 32-bit NVIDIA driver libraries with -nv*. 2024. URL: <https://github.com/apptainer/apptainer/issues/2343> (visited on 03/20/2026).
- [3] Kai Arulkumaran et al. “Deep reinforcement learning: A brief survey”. In: *IEEE signal processing magazine* 34.6 (2017), pp. 26–38.
- [4] Ibadi (Leonel Badi). *Error running without physical display, but using xvfb*. 2023. URL: <https://github.com/doitsujin/dxvk/issues/3706> (visited on 03/20/2026).
- [5] Marc G. Bellemare, Will Dabney, and Rémi Munos. *A Distributional Perspective on Reinforcement Learning*. 2017. arXiv: 1707.06887 [cs.LG]. URL: <https://arxiv.org/abs/1707.06887>.
- [6] Yakir Berchenko. “Simplicity bias in overparameterized machine learning”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 38. 10. 2024, pp. 11052–11060.
- [7] Aditya Bhatt et al. *CrossQ: Batch Normalization in Deep Reinforcement Learning for Greater Sample Efficiency and Simplicity*. 2024. arXiv: 1902.05605 [cs.LG]. URL: <https://arxiv.org/abs/1902.05605>.
- [8] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*. Vol. 4. 4. Springer, 2006.
- [9] Leiyu Chen et al. “Review of Image Classification Algorithms Based on Convolutional Neural Networks”. In: *Remote Sensing* 13 (Nov. 2021), p. 4712. DOI: 10.3390/rs13224712.
- [10] Richard Cheng et al. *Control Regularization for Reduced Variance Reinforcement Learning*. 2019. arXiv: 1905.05380 [cs.LG]. URL: <https://arxiv.org/abs/1905.05380>.
- [11] Po-Wei Chou, Daniel Maturana, and Sebastian Scherer. “Improving Stochastic Policy Gradients in Continuous Control with Deep Reinforcement Learning using the Beta Distribution”. In: *Proceedings of the 34th International Conference on Machine Learning*. Ed. by Doina Precup and Yee Whye Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, June 2017, pp. 834–843. URL: <https://proceedings.mlr.press/v70/chou17a.html>.
- [12] Kurtland Chua et al. *Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models*. 2018. arXiv: 1805.12114 [cs.LG]. URL: <https://arxiv.org/abs/1805.12114>.
- [13] Nicholas E. Corrado and Josiah P. Hanna. *On-Policy Policy Gradient Reinforcement Learning Without On-Policy Sampling*. 2024. arXiv: 2311.08290 [cs.LG]. URL: <https://arxiv.org/abs/2311.08290>.
- [14] Will Dabney et al. *Distributional Reinforcement Learning with Quantile Regression*. 2017. arXiv: 1710.10044 [cs.AI]. URL: <https://arxiv.org/abs/1710.10044>.
- [15] Will Dabney et al. *Implicit Quantile Networks for Distributional Reinforcement Learning*. 2018. arXiv: 1806.06923 [cs.LG]. URL: <https://arxiv.org/abs/1806.06923>.
- [16] Mostafa Dehghani et al. “Scaling Vision Transformers to 22 Billion Parameters”. In: 2023. URL: <https://openreview.net/forum?id=Lhy8H75KA>.
- [17] Marc Peter Deisenroth, Dieter Fox, and Carl Edward Rasmussen. “Gaussian Processes for Data-Efficient Learning in Robotics and Control”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 37.2 (2015), pp. 408–423. DOI: 10.1109/TPAMI.2013.218.
- [18] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. arXiv: 1810.04805 [cs.CL]. URL: <https://arxiv.org/abs/1810.04805>.

- [19] Shibhansh Dohare et al. *Maintaining Plasticity in Deep Continual Learning*. 2024. arXiv: 2306.13812 [cs.LG]. URL: <https://arxiv.org/abs/2306.13812>.
- [20] donadigo#6525. *tminterface*. <https://donadigo.com/tminterface/>. Accessed: 17 August 2025. n.d.
- [21] Alexey Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV]. URL: <https://arxiv.org/abs/2010.11929>.
- [22] Jesse Farebrother et al. *Stop Regressing: Training Value Functions via Classification for Scalable Deep RL*. 2024. arXiv: 2403.03950 [cs.LG]. URL: <https://arxiv.org/abs/2403.03950>.
- [23] Andreas Folkers, Matthias Rick, and Christof Buskens. “Controlling an Autonomous Vehicle with Deep Reinforcement Learning”. In: *2019 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, June 2019, pp. 2025–2031. DOI: 10.1109/ivs.2019.8814124. URL: <http://dx.doi.org/10.1109/IVS.2019.8814124>.
- [24] Florian Fuchs et al. “Super-Human Performance in Gran Turismo Sport Using Deep Reinforcement Learning”. In: *IEEE Robotics and Automation Letters* 6.3 (July 2021), pp. 4257–4264. ISSN: 2377-3774. DOI: 10.1109/lra.2021.3064284. URL: <http://dx.doi.org/10.1109/LRA.2021.3064284>.
- [25] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [26] Shixiang Gu et al. *Q-Prop: Sample-Efficient Policy Gradient with An Off-Policy Critic*. 2017. arXiv: 1611.02247 [cs.LG]. URL: <https://arxiv.org/abs/1611.02247>.
- [27] Tuomas Haarnoja et al. *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. 2018. arXiv: 1801.01290 [cs.LG]. URL: <https://arxiv.org/abs/1801.01290>.
- [28] Peter Henderson et al. *Deep Reinforcement Learning that Matters*. 2019. arXiv: 1709.06560 [cs.LG]. URL: <https://arxiv.org/abs/1709.06560>.
- [29] Jian Hu et al. *BroRL: Scaling Reinforcement Learning via Broadened Exploration*. 2025. arXiv: 2510.01180 [cs.LG]. URL: <https://arxiv.org/abs/2510.01180>.
- [30] Ehsan Imani and Martha White. *Improving Regression Performance with Distributional Losses*. 2018. arXiv: 1806.04613 [stat.ML]. URL: <https://arxiv.org/abs/1806.04613>.
- [31] Ehsan Imani et al. *Investigating the Histogram Loss in Regression*. 2024. arXiv: 2402.13425 [cs.LG]. URL: <https://arxiv.org/abs/2402.13425>.
- [32] Jaeik Jeong, Tai-Yeon Ku, and Wan-Ki Park. “Exploring the Preference for Discrete over Continuous Reinforcement Learning in Energy Storage Arbitrage”. In: *Energies* 17.23 (2024). ISSN: 1996-1073. DOI: 10.3390/en17235876. URL: <https://www.mdpi.com/1996-1073/17/23/5876>.
- [33] Lukasz Kaiser et al. *Model-Based Reinforcement Learning for Atari*. 2024. arXiv: 1903.00374 [cs.LG]. URL: <https://arxiv.org/abs/1903.00374>.
- [34] Vijay Konda and John Tsitsiklis. “Actor-critic algorithms”. In: *Advances in neural information processing systems* 12 (1999).
- [35] Pius Kriemler. *Distributed Setup for Trackmania Reinforcement Learning*. <https://pub.tik.ee.ethz.ch/students/2023-FS/GA-2023-03.pdf>. Bachelor’s Thesis, ETH Zurich. 2023.
- [36] Aviral Kumar et al. “DR3: Value-Based Deep Reinforcement Learning Requires Explicit Regularization”. In: *International Conference on Learning Representations*. 2022. URL: <https://openreview.net/forum?id=POvMvLi91f>.
- [37] Hojoon Lee et al. “A Champion-Level Vision-Based Reinforcement Learning Agent for Competitive Racing in Gran Turismo 7”. In: *IEEE Robotics and Automation Letters* 10.6 (2025), pp. 5545–5552. DOI: 10.1109/LRA.2025.3560873.
- [38] Hojoon Lee et al. *Hyperspherical Normalization for Scalable Deep Reinforcement Learning*. 2025. arXiv: 2502.15280 [cs.LG]. URL: <https://arxiv.org/abs/2502.15280>.
- [39] Hojoon Lee et al. *SimBa: Simplicity Bias for Scaling Up Parameters in Deep Reinforcement Learning*. 2025. arXiv: 2410.09754 [cs.LG]. URL: <https://arxiv.org/abs/2410.09754>.
- [40] Qiyang Li, Zhiyuan Zhou, and Sergey Levine. *Reinforcement Learning with Action Chunking*. 2025. arXiv: 2507.07969 [cs.LG]. URL: <https://arxiv.org/abs/2507.07969>.
- [41] Clare Lyle, Mark Rowland, and Will Dabney. *Understanding and Preventing Capacity Loss in Reinforcement Learning*. 2022. arXiv: 2204.09560 [cs.LG]. URL: <https://arxiv.org/abs/2204.09560>.
- [42] Girolamo Macaluso, Alessandro Sestini, and Andrew D. Bagdanov. *A Benchmark Environment for Offline Reinforcement Learning in Racing Games*. 2024. arXiv: 2407.09415 [cs.AI]. URL: <https://arxiv.org/abs/2407.09415>.
- [43] Warwick Masson, Praveesh Ranchod, and George Konidaris. *Reinforcement Learning with Parameterized Actions*. 2015. arXiv: 1509.01644 [cs.AI]. URL: <https://arxiv.org/abs/1509.01644>.
- [44] Volodymyr Mnih et al. *Playing Atari with Deep Reinforcement Learning*. 2013. arXiv: 1312.5602 [cs.LG]. URL: <https://arxiv.org/abs/1312.5602>.
- [45] L.J. Neinders. *Improving Trackmania Reinforcement Learning Performance : A Comparison of Sophy and Trackmania AI*. May 2023. URL: <http://essay.utwente.nl/96153/>.
- [46] Behnam Neyshabur et al. *Towards Understanding the Role of Over-Parametrization in Generalization of Neural Networks*. 2018. arXiv: 1805.12076 [cs.LG]. URL: <https://arxiv.org/abs/1805.12076>.

- [47] Evgenii Nikishin et al. *The Primacy Bias in Deep Reinforcement Learning*. 2022. arXiv: 2205.07802 [cs.LG]. URL: <https://arxiv.org/abs/2205.07802>.
- [48] Seohong Park, Qiyang Li, and Sergey Levine. *Flow Q-Learning*. 2025. arXiv: 2502.02538 [cs.LG]. URL: <https://arxiv.org/abs/2502.02538>.
- [49] Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.
- [50] Antonin Raffin et al. “Stable-Baselines3: Reliable Reinforcement Learning Implementations”. In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8. URL: <http://jmlr.org/papers/v22/20-1364.html>.
- [51] Adrian Remonda et al. *Formula RL: Deep Reinforcement Learning for Autonomous Racing using Telemetry Data*. 2022. arXiv: 2104.11106 [cs.AI]. URL: <https://arxiv.org/abs/2104.11106>.
- [52] Julian Schrittwieser et al. “Mastering Atari, Go, chess and shogi by planning with a learned model”. In: *Nature* 588.7839 (Dec. 2020), pp. 604–609. ISSN: 1476-4687. DOI: 10.1038/s41586-020-03051-4. URL: <http://dx.doi.org/10.1038/s41586-020-03051-4>.
- [53] John Schulman et al. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. 2018. arXiv: 1506.02438 [cs.LG]. URL: <https://arxiv.org/abs/1506.02438>.
- [54] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. arXiv: 1707.06347 [cs.LG]. URL: <https://arxiv.org/abs/1707.06347>.
- [55] John Schulman et al. *Trust Region Policy Optimization*. 2017. arXiv: 1502.05477 [cs.LG]. URL: <https://arxiv.org/abs/1502.05477>.
- [56] Harshay Shah et al. “The pitfalls of simplicity bias in neural networks”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 9573–9585.
- [57] David Silver et al. *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. 2017. arXiv: 1712.01815 [cs.AI]. URL: <https://arxiv.org/abs/1712.01815>.
- [58] Tahmur Hassan Mahmood Simon Wilhelm Schübel. *trackmania-gym: A Gymnasium compatible Reinforcement Learning Framework for Trackmania Nations Forever*. <https://github.com/dersiwi/trackmania-gym>. 2024.
- [59] Lawrence Stewart et al. *Regression as Classification: Influence of Task Formulation on Neural Network Features*. 2023. arXiv: 2211.05641 [cs.LG]. URL: <https://arxiv.org/abs/2211.05641>.
- [60] Richard S Sutton et al. “Policy gradient methods for reinforcement learning with function approximation”. In: *Advances in neural information processing systems* 12 (1999).
- [61] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. Cambridge, MA: MIT Press, 2018. URL: <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>.
- [62] Mark Towers et al. *Gymnasium: A Standard Interface for Reinforcement Learning Environments*. 2024. arXiv: 2407.17032 [cs.LG]. URL: <https://arxiv.org/abs/2407.17032>.
- [63] Hado Van Hasselt. “Reinforcement Learning in Continuous State and Action Spaces”. In: (Aug. 2013). DOI: 10.1007/978-3-642-27645-3_7.
- [64] Tingwu Wang et al. *Benchmarking Model-Based Reinforcement Learning*. 2019. arXiv: 1907.02057 [cs.LG]. URL: <https://arxiv.org/abs/1907.02057>.
- [65] Ziyu Wang et al. *Dueling Network Architectures for Deep Reinforcement Learning*. 2016. arXiv: 1511.06581 [cs.LG]. URL: <https://arxiv.org/abs/1511.06581>.
- [66] Christopher JCH Watkins and Peter Dayan. “Q-learning”. In: *Machine learning* 8.3 (1992), pp. 279–292.
- [67] Ronald J Williams. “Simple statistical gradient-following algorithms for connectionist reinforcement learning”. In: *Machine learning* 8.3 (1992), pp. 229–256.
- [68] Hongxiang Yang, Lili Zhu, and Yingkun Hou. “Research on Computer Vision Image Classification System Based on Deep Learning”. In: *2023 International Conference on Applied Physics and Computing (ICAPC)*. 2023, pp. 586–590. DOI: 10.1109/ICAPC61546.2023.00115.
- [69] Chiyuan Zhang et al. *Understanding deep learning requires rethinking generalization*. 2017. arXiv: 1611.03530 [cs.LG]. URL: <https://arxiv.org/abs/1611.03530>.

APPENDIX A TRAINING ENVIRONMENTS



A01



A02



B03

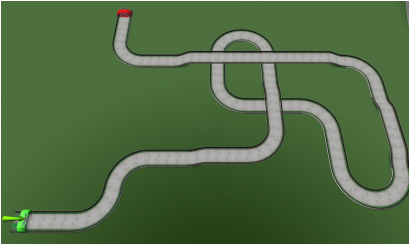


C01

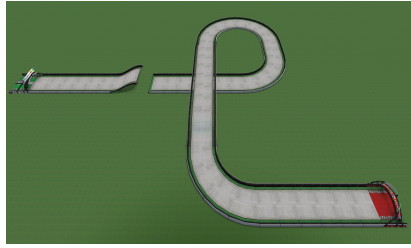


C02

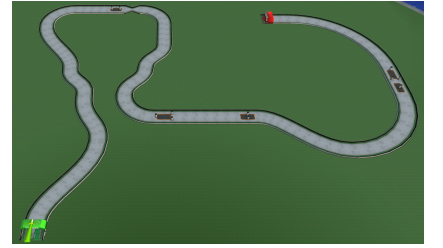
Fig. 10: **Official TMNF Maps.** Overview of the official TMNF tracks utilized for training and evaluation within this work.



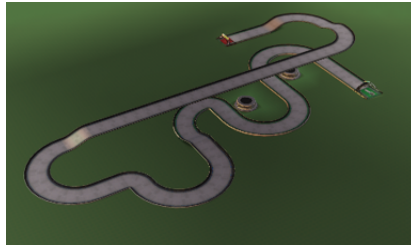
Another1



Another2



Another3



Simple1

Fig. 11: **Custom Racing Environments.** Overview of custom track layouts built specifically for this work.

TABLE I: **Episode step limits per environment track.** These values were derived by measuring the number of steps required for a human driver to complete each track and applying a multiplier of three. This buffer ensures that early sub-optimal policies have sufficient time to reach the goal.

Map Name	A01	A02	B03	C01	C02	Anoth.1	Anoth.2	Anoth.3	Simpl.1
Max Steps	2449	1800	2049	3499	4800	2049	1800	2049	2049

APPENDIX B HYPERPARAMETERS

TABLE II: **Hyperparameter configurations for PPO and SimbaV2-PPO.** Shared parameters are kept identical to ensure a fair baseline comparison.

	Hyperparameter	Notation	Value
Common	Learning Rate	α	2.5×10^{-4}
	Number of Steps	T	2048
	Minibatch Size	M	512
	Number of Epochs	K	5
	Discount Factor	γ	0.99
	GAE Parameter	λ	0.95
	Clipping Parameter	ϵ	0.2
	Value Function Coeff.	c_1	1.0
	Entropy Coefficient	c_2	0.01
	Gradient Norm Clipping	$\ g\ $	0.5
	Optimizer	-	Adam
	Optimizer momentum	(β_1, β_2)	(0.9, 0.999)
	Optimizer eps	-	$1e-8$
	Weight decay	-	0
SimbaV2	Number of Atoms (Bins)	N	101
	Support (Min Value)	$V_{\min}$	-5.0
	Support (Max Value)	$V_{\max}$	5.0

TABLE III: **Network architecture specifications for the SimbaV2-PPO agent.** Following the original SimbaV2 framework [38], the critic utilizes a significantly deeper configuration. For exhaustive implementation details regarding layer initialization and residual connections, the reader is referred to the original work.

	Layer / Block	Hidden Dimension
Actor	Block 1	128
	Block 2	512
	Block 3	128
Critic	Block 1	128
	Block 2	512
	Block 3	2048
	Block 4	512
	Block 5	2048
	Block 6	512
	Block 7	512

TABLE IV: **Architecture of the Vision Encoder used in this works SimbaV2 runs.** The input consists of a $128 \times 128 \times 1$ grayscale image. Note that UnitConv2D refers to a standard Conv2D layer where the kernel weights are consistently normalized to unit length.

Layer Description	Input Dimensions	Output Dimensions
UnitConv2D: 32 filters, 5x5, s=1, p=2, ReLU	$128 \times 128 \times 1$	$128 \times 128 \times 32$
MaxPool2d: 2x2, s=2	$128 \times 128 \times 32$	$64 \times 64 \times 32$
UnitConv2D: 64 filters, 5x5, s=1, p=2, ReLU	$64 \times 64 \times 32$	$64 \times 64 \times 64$
MaxPool2d: 2x2, s=2	$64 \times 64 \times 64$	$32 \times 32 \times 64$
UnitConv2D: 128 filters, 3x3, s=1, p=1, ReLU	$32 \times 32 \times 64$	$32 \times 32 \times 128$
MaxPool2d: 2x2, s=2	$32 \times 32 \times 128$	$16 \times 16 \times 128$
UnitConv2D: 128 filters, 3x3, s=1, p=1, ReLU	$16 \times 16 \times 128$	$16 \times 16 \times 128$
AdaptiveAvgPool2d: (1x1) output	$16 \times 16 \times 128$	$1 \times 1 \times 128$
Flatten	$1 \times 1 \times 128$	128

TABLE V: **Architectural specifications of the baseline Vision Encoder.** The input is a $128 \times 128 \times 1$ grayscale image.

Layer Description	Input Dimensions	Output Dimensions
Conv2d: 32 filters, 5x5, s=1, p=2, ReLU	$128 \times 128 \times 1$	$128 \times 128 \times 32$
MaxPool2d: 2x2, s=2	$128 \times 128 \times 32$	$64 \times 64 \times 32$
Conv2d: 64 filters, 5x5, s=1, p=2, ReLU	$64 \times 64 \times 32$	$64 \times 64 \times 64$
MaxPool2d: 2x2, s=2	$64 \times 64 \times 64$	$32 \times 32 \times 64$
Conv2d: 128 filters, 3x3, s=1, p=1, ReLU	$32 \times 32 \times 64$	$32 \times 32 \times 128$
MaxPool2d: 2x2, s=2	$32 \times 32 \times 128$	$16 \times 16 \times 128$
Conv2d: 128 filters, 3x3, s=1, p=1, ReLU	$16 \times 16 \times 128$	$16 \times 16 \times 128$
Flatten	$16 \times 16 \times 128$	32,768

TABLE VI: **State-space observation vector and normalization factors.** The normalization factor α is not used when using SimbaV2

	Observation Name	Dim	Norm. Factor (α)
Kinematics	Distance passed	1	1
	Vehicle speed	1	1000
	Lateral distance to centerline	1	18
	Upcoming reference points	$3 \times k$	500
Tires & Physics	Wheel sliding	4	1
	Surface material type	4	4
	Ground contact status	4	1
	Damper absorption	4	1
	Freewheeling status	1	1
Drivetrain	Gearbox state	1	5
	Current gear	1	2
	Engine RPM	1	12000

TABLE VII: **Reward weights.** The reward weights for individual reward terms used in this work.

	Hyperparameter	Notation	Value
Success	Race finished weight	w_g	300
Progress	Accum. distance weight	w_a	100
	Exponential factor	η	1.2
Penalties	Termination punishment	w_s	-5
	Wall contact weight	w_w	-0.15

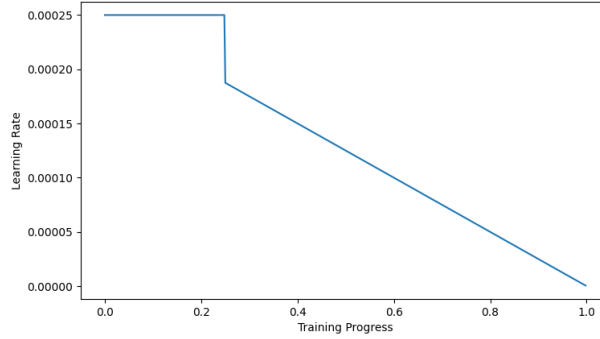


Fig. 12: Visualization of the learning rate over training progress p , which is the number of taken environment steps divided by the total amount of environment steps. The learningrate is constant as long as $p < \varrho$, and once $\varrho \geq p$, the learning rate is scheduled linearly like $\text{lr}_\beta(\varrho) = 2.5 \cdot 10^{-4} \cdot \varrho$ where in this visualization $\varrho = 0.25$.

APPENDIX C
ENVIRONMENT SPECIFICS

A. Visualization of Environmentsteps versus Simulationsteps

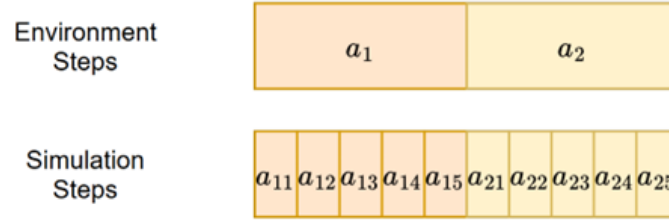


Fig. 13: Visualization of difference in Simulation and Environment steps, showing how an action a_i can be turned into k actions a_{ij} , $j \in \{1, k\}$ where k is the number of simulation steps between each environment step.

APPENDIX D ADDITIONAL EXPERIMENT RESULTS

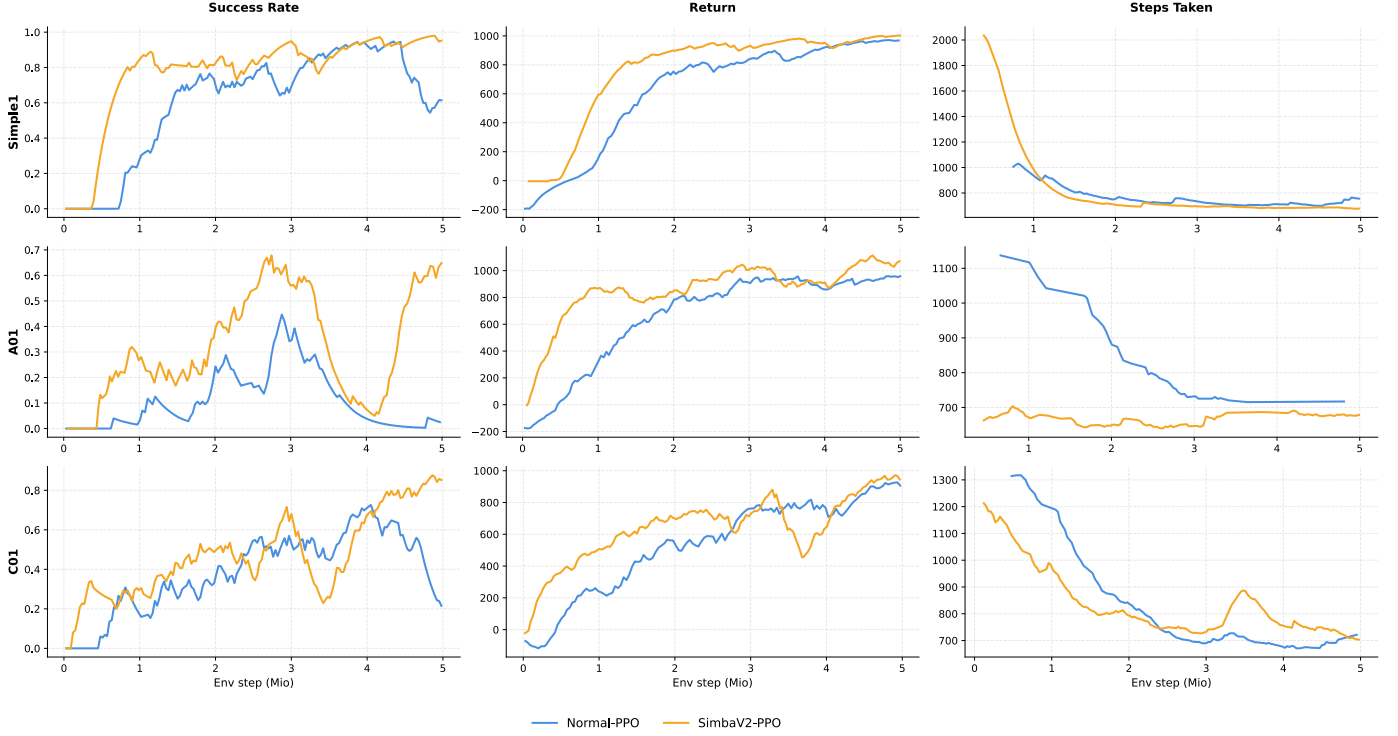


Fig. 14: **Performance comparison between SimBaV2-PPO and standard PPO.** Results are shown across three environments Simple1, A01, and C01. The evaluation metrics include Success Rate, Mean Return, and average Steps Taken to Finish. All metrics are plotted against the number of environment steps. Data is smoothed to highlight learning trends.

A. Sparse Reward with Pretrained SimbaV2 Model

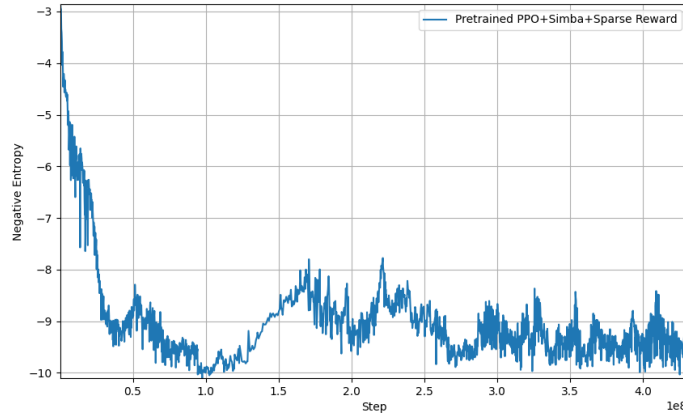


Fig. 15: **Negative Entropy of Sparse Reward.** This plot shows the entropy loss, which, per our implementation, is directly proportional to the negative entropy.

B. Multi-Track per Environment Normalization

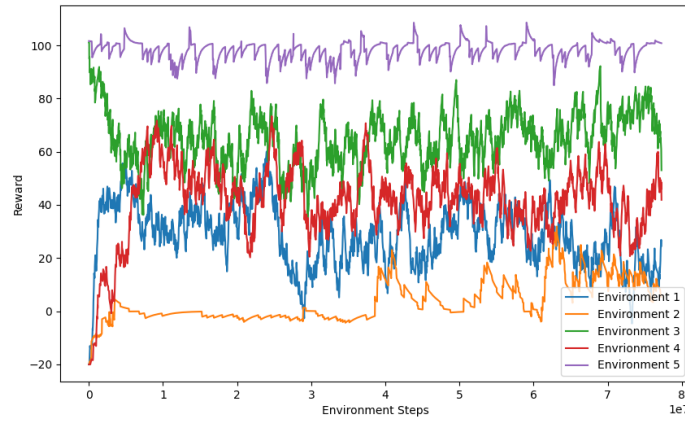


Fig. 16: Total reward in Multi-Track Training of a Regular PPO agent, which is normalized per environment.

C. Multi-Track SimbaV2

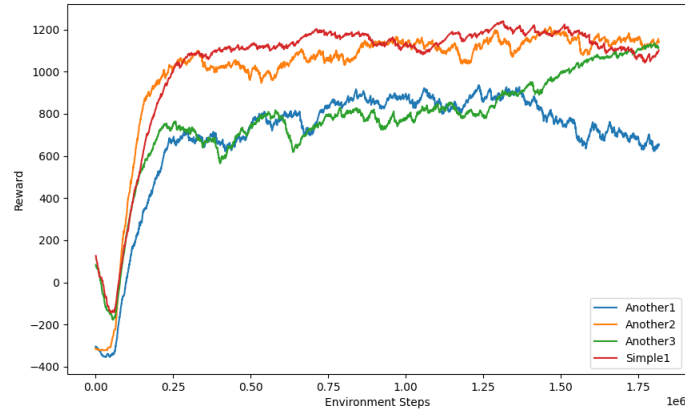


Fig. 17: Total reward of all environments of a SimbaV2 agent trained on Simple1, Another1, Another2, Another3.

D. Multi-track vs Single track performance

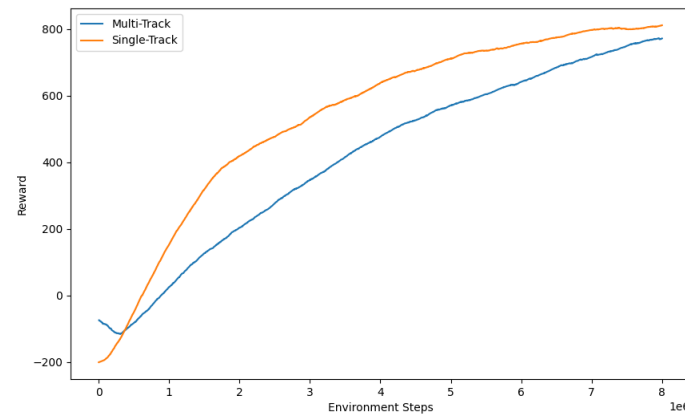


Fig. 18: Reward during evaluation of two SimbaV2 agents, trained on Simple1. The Multi-Track agent was trained on Simple1, Another1-3 at the same time.

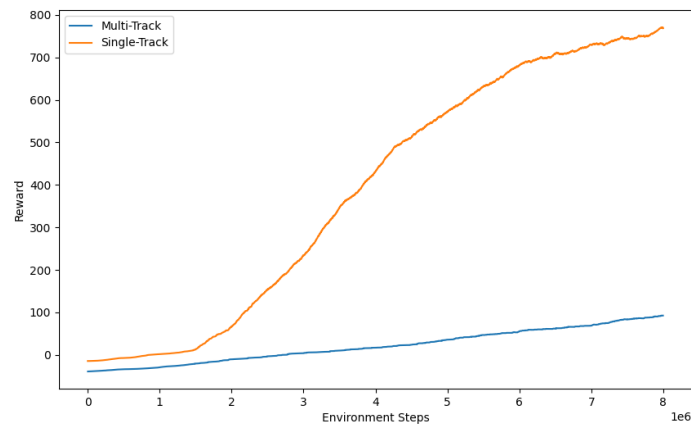


Fig. 19: **Reward during evaluation of two SimbaV2 agents, trained on B03.** The Multi-Track agent was trained on A02, B01, C01, C03 at the same time as B03, the Single-Track agent was only trained on B03.